

STAR Robot Control System

Comprehensive Technical Documentation

Complete Reference for Software Engineers

Version 1.0
December 2025

STAR Development Team
Locked Inc.

Contents

1	Nanopb (Protobuf) Protocol with ARQ and CRC for SPI Communication	2
1.1	Executive Summary	2
1.2	Key Design Decisions	2
1.2.1	Protocol Stack	2
1.2.2	Memory Budget	3
1.3	Implementation Requirements	4
1.3.1	Protocol Stack Architecture	4
1.3.2	ESP32 (SPI Peripheral) Implementation	4
1.3.3	RPi5 (SPI Controller) Implementation	5
1.3.4	ARQ Symmetry and State Management	6
1.3.5	Example Transaction: RPi5 Sends Velocity Command	6
1.4	ARQ Communication Flow	7
1.4.1	Normal Operation (No Retransmission)	7
1.4.2	Error Recovery Scenarios	8
1.5	Error Handling Strategies	9

1.2.2 Memory Budget

Table 1: SRAM Usage (512 KB total)

Component	Size (KB)	Usage (%)
DMA double-buffer	8.0	1.56
RX/TX buffers	2.0	0.39
Protobuf structs	1.5	0.29
ARQ state	4.5	0.88
Total	16.0	3.12

Table 2: Flash Usage (16 MB total)

Component	Size (KB)	Usage (%)
Generated protobuf	40	0.24
Nanopb runtime	12	0.07
Frame + ARQ layers	8	0.05
Total	60	0.37

Memory Budget Calculations:

SRAM (512 KB total):

- **DMA double-buffer:** 2×4092 bytes = 8184 bytes = 8.0 KB
Calculation: Two ping-pong DMA buffers at `STAR_SPI_DMA_MAX_SIZE` (4092 bytes each)
Usage: $\frac{8.0}{512} \times 100 = 1.56\%$
- **RX/TX buffers:** $1024 + 1024 = 2048$ bytes = 2.0 KB
Calculation: 1 KB RX buffer + 1 KB TX buffer for staging protobuf messages
Usage: $\frac{2.0}{512} \times 100 = 0.39\%$
- **Protobuf structs:** 1536 bytes = 1.5 KB
Calculation: Largest message structures (SetVelocityResponse $\approx$ 600 bytes, TelemetryData $\approx$ 400 bytes)
Usage: $\frac{1.5}{512} \times 100 = 0.29\%$
- **ARQ state:** $4 + 4 + 8 + 4 + 4092 = 4112$ bytes $\approx$ 4.5 KB
Calculation: Sequence number (4B) + retry counter (4B) + timestamp (8B) + state (4B) + retry buffer (4092B)
Usage: $\frac{4.5}{512} \times 100 = 0.88\%$
- **Total SRAM:** $8.0 + 2.0 + 1.5 + 4.5 = 16.0$ KB
Total usage: $\frac{16.0}{512} \times 100 = 3.12\%$

Flash (16 MB = 16384 KB total):

- **Generated protobuf:** 40 KB
Calculation: 3 proto files (common.proto, motor_control.proto, telemetry.proto) $\times$ 13 KB avg compiled size
Usage: $\frac{40}{16384} \times 100 = 0.24\%$
- **Nanopb runtime:** 12 KB
Calculation: Compiled size of pb_encode.c, pb_decode.c, pb_common.c
Usage: $\frac{12}{16384} \times 100 = 0.07\%$
- **Frame + ARQ layers:** 8 KB
Calculation: Framing logic + CRC-32 + ARQ state machine + retransmission logic
Usage: $\frac{8}{16384} \times 100 = 0.05\%$
- **Total Flash:** $40 + 12 + 8 = 60$ KB
Total usage: $\frac{60}{16384} \times 100 = 0.37\%$

1.3 Implementation Requirements

1.3.1 Protocol Stack Architecture

The protocol is implemented as a layered stack on both the RPi5 (SPI controller) and ESP32 (SPI peripheral). Each layer has specific responsibilities:

Layer	Responsibility	Implementation
5. Application	Motor commands, telemetry, control logic	Different per platform
4. Protobuf	Message encode/decode (nanopb)	Both sides
3. ARQ	Sequence numbering, ACK/NACK, retry logic	Both sides (symmetric)
2. Framing	Header/footer, CRC-32 verification	Both sides (identical)
1. SPI+DMA	Physical transport with double-buffering	Both sides (different roles)

Table 3: Protocol stack layers and implementation requirements

Key Insight: Layers 2-4 are nearly identical on both platforms. Only the application layer (Layer 5) and SPI role (Layer 1: controller vs peripheral) differ.

1.3.2 ESP32 (SPI Peripheral) Implementation

Layer 1: SPI Peripheral with DMA

- Configure SPI3 in peripheral mode using ESP-IDF
- Set up DMA with double-buffering: 2×4092 bytes
- Register DMA interrupt callbacks for buffer swapping
- Wait for chip select from RPi5, respond to clock

Layer 2: Frame Layer (Custom Code)

RX Path:

- Parse frame header (magic bytes, length, sequence number)
- Verify CRC-32 over header + payload
- If CRC fails $\rightarrow$ discard packet, send NACK
- If CRC valid $\rightarrow$ extract payload, pass to ARQ layer

TX Path:

- Build frame: [Header | Payload | CRC32 | Footer]
- Calculate CRC-32 over header + payload
- Queue complete frame to DMA buffer for transmission

Layer 3: ARQ Layer (Custom Code)

RX Path:

- Check sequence number against `last_acked_rx`
- If duplicate (`seq ≤ last_acked_rx`) → send ACK, discard payload
- If expected (`seq = last_acked_rx + 1`) → accept, send ACK
- If out-of-order → send NACK, request retransmission

TX Path:

- Assign `tx_sequence_number` to outgoing frame
- Store frame in retry buffer
- Start timeout timer (e.g., 10ms)
- On timeout → retransmit (up to 3 attempts)
- On ACK received → clear retry buffer, increment `tx_seq`

Layer 4: Protobuf (nanopb library)

- Decode incoming bytes → `SetVelocityRequest`, `EmergencyStopRequest`
- Encode outgoing structs → `SetVelocityResponse`, `TelemetryData`
- Use static allocation (no malloc) with `.options` file constraints

Layer 5: Application

- Handle motor velocity commands
- Read encoder data via multiplexer
- Execute PID control loop (250 Hz per motor)
- Send telemetry responses (encoder feedback, current, temperature)

1.3.3 RPi5 (SPI Controller) Implementation

Layer 1: SPI Controller with DMA

- Use Linux `/dev/spidev` interface
- Configure SPI3: 10 Mbps, mode 0, 8-bit words
- Use `ioctl(SPI_IOC_MESSAGE)` for DMA transfers
- Implement userspace double-buffering for continuous operation

Layer 2: Frame Layer (*Custom Code - SAME AS ESP32*)

RX Path: Parse frame header, verify CRC-32, extract payload or discard on CRC failure

TX Path: Build frame [`Header` | `Payload` | `CRC32` | `Footer`], calculate CRC-32, send via SPI

Layer 3: ARQ Layer (*Custom Code - SAME AS ESP32*)

RX Path: Check sequence number, detect duplicates → ACK and discard, detect out-of-order → NACK

TX Path: Assign sequence number, store in retry buffer, timeout and retry logic, handle ACKs

Layer 4: Protobuf (nanopb, protobuf-c, or gRPC)

- Encode: SetVelocityRequest, EmergencyStopRequest
- Decode: SetVelocityResponse, TelemetryData, EncoderData

Layer 5: Application

- Send motor velocity commands (from Nav2 or joystick)
- Receive encoder feedback for odometry
- Integrate with ROS2 navigation stack
- Handle UI commands and visualization

1.3.4 ARQ Symmetry and State Management

The ARQ protocol is **symmetric** — both sides implement identical logic but maintain separate state for their own transmissions and receptions.

Each Side Maintains:

- **TX sequence number** (`tx_seq`): For packets it sends
- **RX last acknowledged** (`rx_last_ack`): For packets it receives
- **Retry buffer**: Stores unacknowledged outgoing packets
- **Timeout timer**: Triggers retransmission if no ACK received

Responsibility	ESP32 (Peripheral)	RPi5 (Controller)
Sends	Telemetry, responses	Commands, requests
TX sequence numbering	Own <code>tx_seq</code>	Own <code>tx_seq</code>
ACK/NACK generation	Sends ACK for RPi5	Sends ACK for ESP32
Retry logic	Retries if no ACK	Retries if no ACK
Duplicate detection	Checks RPi5's <code>seq</code>	Checks ESP32's <code>seq</code>

Table 4: ARQ role distribution between controller and peripheral

Important: The SPI controller/peripheral roles (Layer 1) are **hardware-level only**. The ARQ layer (Layer 3) treats both sides as equals — each can send, ACK, and retry independently.

1.3.5 Example Transaction: RPi5 Sends Velocity Command

1. **RPi5 Application:** Create SetVelocityRequest (left=1.0 m/s, right=1.0 m/s)
2. **RPi5 Protobuf:** Encode to binary (nanopb)
3. **RPi5 ARQ:** Assign `seq`=42, save to retry buffer, start 10ms timeout
4. **RPi5 Frame:** Build packet with header, CRC-32, footer
5. **RPi5 SPI:** DMA transfer to ESP32 (controller initiates)
6. **ESP32 SPI:** DMA receives packet in double-buffer

7. **ESP32 Frame:** Verify CRC-32 $\rightarrow$ valid
8. **ESP32 ARQ:** Check $\text{seq}=42 = \text{rx_last_ack}+1 \rightarrow$ accept
9. **ESP32 Protobuf:** Decode to `SetVelocityRequest` struct
10. **ESP32 Application:** Update motor setpoints, execute PID control
11. **ESP32 ARQ:** Build ACK packet with $\text{ack_seq}=42$
12. **ESP32 Frame:** Wrap ACK in frame with CRC-32
13. **ESP32 SPI:** DMA transmits ACK (peripheral responds)
14. **RPi5 SPI:** DMA receives ACK packet
15. **RPi5 Frame:** Verify CRC-32 $\rightarrow$ valid
16. **RPi5 ARQ:** Got ACK(42) $\rightarrow$ clear retry buffer, cancel timeout

Outcome: Command successfully delivered with guaranteed delivery via ARQ. If ACK was lost or CRC failed, RPi5 would retransmit after 10ms timeout.

1.4 ARQ Communication Flow

1.4.1 Normal Operation (No Retransmission)

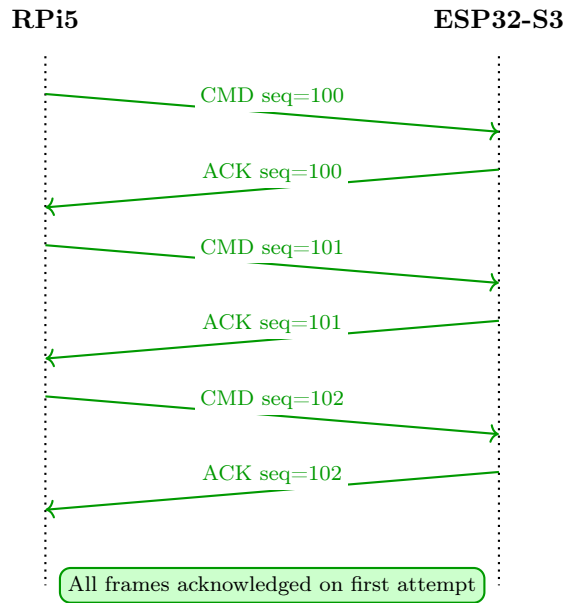


Figure 1: Normal ARQ Flow - Three Successful Transactions

1.4.2 Error Recovery Scenarios

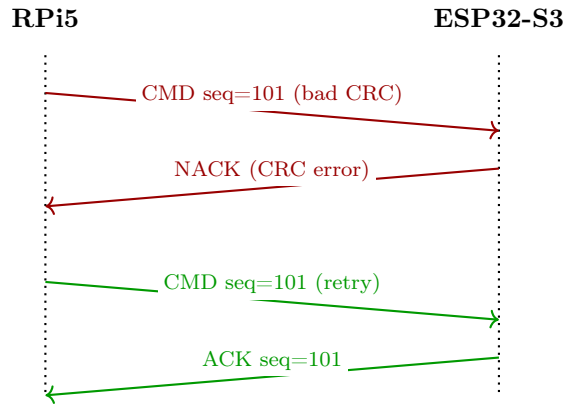


Figure 2: CRC Error - Retry with Same Sequence Number

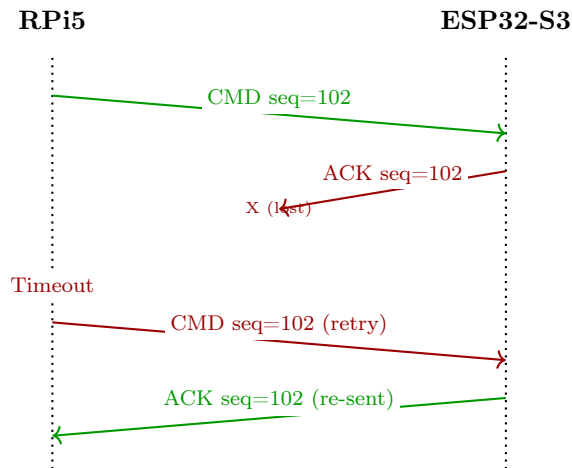


Figure 3: Duplicate Packet Scenario (Lost ACK)

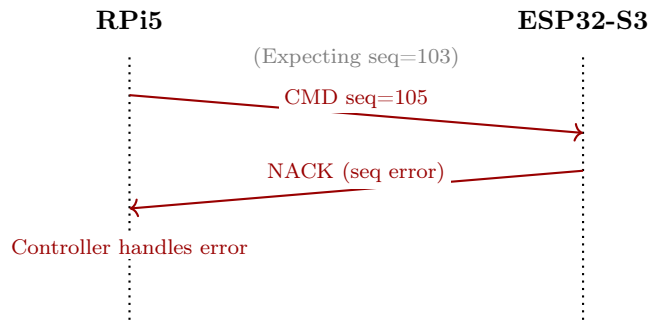


Figure 4: Out-of-Order Packet Scenario

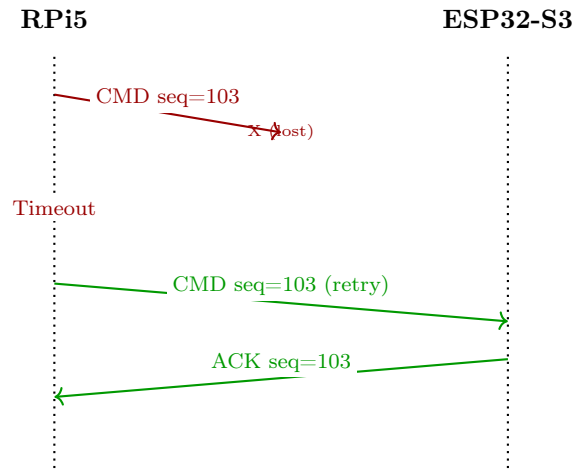


Figure 5: Lost Packet Scenario (Timeout)

1.5 Error Handling Strategies

CRC Mismatch

Action: NACK with CRC error reason
Recovery: Controller retry
Logging: Increment `crc_errors`

Sequence Error

Duplicate: Re-send ACK
Out-of-order: NACK + reject
Recovery: Controller retry

Timeout

RX timeout: Continue next cycle
500ms watchdog: Emergency stop
Retry fail: Log error

Invalid Protobuf

Action: NACK protocol error
Validate: IDs, ranges, offsets
Logging: Increment `proto_errors`